

Java Backend Architecture

Interview Series

Chapter 16.4

Docker Networking

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Chapter 16.4 – Docker Networking

Docker Networking Overview

Docker provides multiple network drivers to support different use cases — from single-host container communication to multi-host overlay networks in Docker Swarm. Understanding network isolation, DNS resolution, and port mapping is essential for production container deployments.

Docker Network Drivers

bridge Default NAT docker0 bridge	host No isolation Host network stack	overlay Multi-host Swarm / K8s	macvlan L2 on host Direct MAC addr	none Loopback only Fully isolated
--	---	---	---	--

Docker Containers

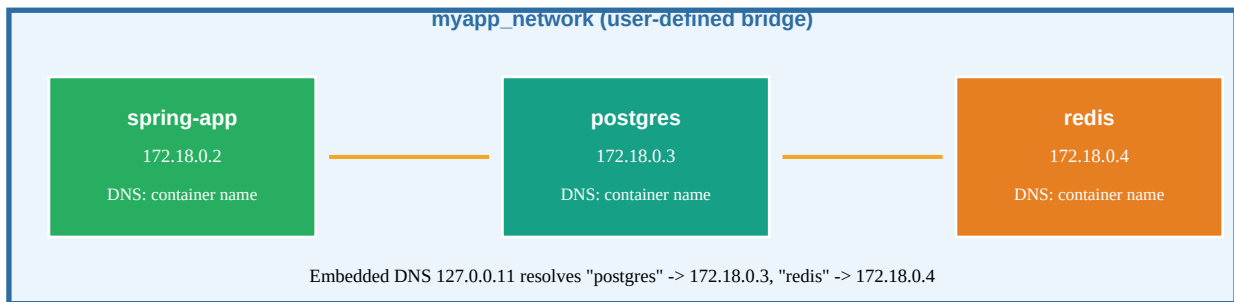
Select network driver per container/service

Docker Network Drivers – Feature Comparison

bridge (default)	Creates docker0 bridge interface. Containers get private IPs (172.17.0.x). NAT via iptables for outbound. Containers cannot reach each other by name (default bridge); must use --link (deprecated) or use user-defined bridge.
user-defined bridge	docker network create mynet. Provides automatic DNS: containers resolve each other by name. Recommended for multi-container apps. Isolated from other bridges.
host	Container shares host network stack. No isolation, no NAT overhead. Best performance. Linux-only (not supported on Docker Desktop Mac/Windows). Use for high-throughput networking.
overlay	Multi-host networking using VXLAN. Required for Docker Swarm services. Containers on different hosts communicate as if on the same network. Supports encryption (--opt encrypted).
macvlan	Assigns a real MAC address to container. Container appears as a physical device on the network. Use when containers need to be directly accessible on L2 network.
none	Container has only loopback interface (127.0.0.1). No network access. Use for maximum isolation (batch jobs, security-sensitive workloads).
ipvlan	Like macvlan but shares MAC address. Uses L2 or L3 mode. Better for environments where MAC address limits apply (cloud hypervisors).

User-Defined Bridge – DNS Resolution

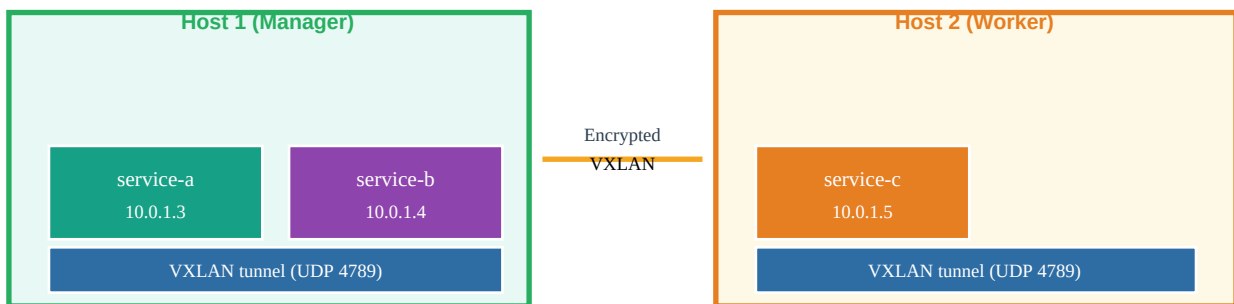
User-Defined Bridge – DNS Resolution



```
# Create user-defined network docker network create myapp_network # Run containers on the
network docker run -d --name postgres --network myapp_network postgres:16 docker run -d --name
redis --network myapp_network redis:7 docker run -d --name spring-app --network myapp_network \
-e SPRING_DATASOURCE_URL=jdbc:postgresql://postgres:5432/mydb \ -e SPRING_REDIS_HOST=redis \
myapp:latest # Containers resolve each other by name via embedded DNS (127.0.0.11)
```

Overlay Multi-Host Communication

Overlay Network – Multi-Host Communication



Overlay subnet: 10.0.1.0/24 — containers communicate across hosts by name

Port Mapping

```
# Map host port 8080 to container port 8080 docker run -p 8080:8080 myapp # Map specific
interface docker run -p 127.0.0.1:8080:8080 myapp # Map to random ephemeral port docker run -p
8080 myapp docker port <container> # see assigned port # Map UDP port docker run -p 53:53/udp
dns-server
```

iptables Rules

Docker automatically manages iptables rules for port mapping and container networking:

```
# View Docker-managed iptables rules iptables -t nat -L -n -v iptables -L DOCKER -n -v #
PREROUTING: DNAT for port mapping # -A PREROUTING -m addrtype --dst-type LOCAL -j DOCKER # -A
DOCKER -p tcp --dport 8080 -j DNAT --to-destination 172.17.0.2:8080 # FORWARD: allow container
traffic # -A FORWARD -i docker0 -o eth0 -j ACCEPT # -A FORWARD -o docker0 -m conntrack
--ctstate RELATED,ESTABLISHED -j ACCEPT
```

Network Troubleshooting with netshoot

```
# Attach netshoot to debug container networking docker run --rm -it --network
container:<container_id> \ nicolaka/netshoot # Inside netshoot: curl -v http://postgres:5432 #
test DNS + TCP connectivity nslookup postgres # DNS resolution check ss -tlnp # listening
sockets tcpdump -i eth0 port 5432 # capture traffic ip route # routing table netstat -r # route
table # Use on host network to inspect docker0 docker run --rm -it --network host
nicolaka/netshoot
```

docker network Commands

```
docker network ls # list networks docker network inspect myapp_network # detailed JSON docker
network create --driver overlay \ # create overlay --attachable myoverlay docker network
connect mynet container1 # connect to network docker network disconnect mynet container1 #
disconnect docker network rm mynet # remove docker network prune # remove all unused
```

Interview Q&A; – Docker Networking

Q1. What is the difference between the default bridge and user-defined bridge networks?

Default bridge (docker0): containers can communicate by IP only (not by name), require `--link` (deprecated) for name resolution, are on the same network as all other default-bridge containers. User-defined bridge: containers resolve each other by name via Docker's embedded DNS server (127.0.0.11), provide better isolation (only containers on the same user-defined network can communicate), and support dynamic connect/disconnect. Always use user-defined bridges for production multi-container applications.

Q2. How does Docker's embedded DNS work?

Docker runs an embedded DNS server at 127.0.0.11 for user-defined networks. Container `/etc/resolv.conf` points to 127.0.0.11. When a container resolves another container's name (e.g. 'postgres'), the DNS server looks up the container name in the network's container registry and returns its IP address. External names are forwarded to the host's DNS resolver. The DNS server is per-network — containers on different user-defined networks cannot resolve each other by name.

Q3. What is the Docker host network driver and when should you use it?

With `--network host`, the container shares the host's network stack directly — no network namespace isolation. The container sees the host's interfaces, IP addresses, and open ports. Benefits: no NAT overhead (best performance for high-throughput apps), no port mapping needed. Drawbacks: no network isolation (container can bind to any host port), Linux-only (not supported on Docker Desktop for Mac/Windows). Use for performance-critical apps like network proxies, monitoring agents, or when latency matters more than isolation.

Q4. Explain Docker overlay networking and when it's used.

Overlay networks span multiple Docker hosts using VXLAN (UDP port 4789). They create a virtual network where containers on different hosts appear on the same subnet. Docker Swarm uses overlay for service-to-service communication. Container traffic is encapsulated in VXLAN packets, transmitted between hosts, and decapsulated. Supports encryption (`--opt encrypted`) using IPsec. Requires Docker Swarm mode (docker swarm init) to create overlay networks, or the `--attachable` flag for standalone containers.

Q5. How does Docker port mapping work at the kernel level?

`docker run -p 8080:8080` creates iptables rules: a PREROUTING DNAT rule that redirects traffic arriving at host:8080 to container_ip:8080, and a FORWARD rule allowing traffic between the host network and docker0 bridge. The docker-proxy process also listens on the host port to handle localhost traffic that bypasses iptables. Port mapping is NAT — the container receives connections on port 8080 but they originate from docker0 gateway IP, not the client IP (use `--net=host` if you need real client IPs).

Q6. What is MACVLAN networking in Docker?

MACVLAN assigns a real MAC address to a container, making it appear as a physical network device on the L2 network. The container gets its own IP directly from the physical network (via DHCP or static), bypassing NAT completely. Use cases: legacy apps that expect to be on the physical network, network monitoring tools that need direct L2 access, environments where containers must have routable IPs visible to the entire network. Requires the host NIC to be in promiscuous mode and network switch support for multiple MACs per port.

Q7. How do you implement service discovery in Docker Compose?

Docker Compose automatically creates a user-defined network for the application. Services resolve each other by their service name as defined in `docker-compose.yml`. If you define service `postgres`, other services reach it at `jdbcp:postgres://postgres:5432/db`. The embedded DNS server (127.0.0.11) handles resolution.

You can also set container aliases: `networks: mynet: aliases: [db, database]`. This allows legacy apps that expect different hostnames to find the same container.

Q8. What is the difference between `-p` and `--expose` in Docker?

`-p` (`--publish`) maps a container port to a host port, making the service accessible from outside the container. `--expose` (or `EXPOSE` in Dockerfile) documents that the container uses a port but does NOT publish it to the host. Exposed ports are accessible to other containers on the same network, but not from the host or external network. `-P` (capital P) publishes all exposed ports to random host ephemeral ports. Summary: `EXPOSE` = internal documentation + inter-container access; `-p` = host-accessible port mapping.

Q9. How do you connect a container to multiple networks?

A container can be connected to multiple networks simultaneously: `docker network connect network2 container1`. Each network gives the container an additional virtual interface (`eth1`, `eth2`, etc.) with a separate IP address. This is useful when a container acts as a gateway or proxy between isolated networks. In Docker Compose, a service can be listed under multiple networks. Container has connectivity to all containers on each network it's connected to, enabling selective isolation.

Q10. What tools do you use to troubleshoot Docker network issues?

Key tools: (1) `nicolaka/netshoot` — debug container with all networking tools (`ping`, `curl`, `tcpdump`, `ss`, `nslookup`). (2) `docker network inspect` — shows container IPs, gateway, IPAM config. (3) `docker exec container cat /etc/resolv.conf` — check DNS config. (4) `docker exec container ss -tlnp` — check listening ports. (5) `iptables -L DOCKER -n -v` — check port mapping rules. (6) `ip route` inside container — routing table. (7) `docker stats` — check if network is saturated. Attach `netshoot` to a running container's network namespace.

Q11. What is `--network container:` in Docker?

`--network container:` shares the network namespace of an existing container. The new container gets the same network interfaces, IP address, and open ports as the target container. This is how sidecars work (e.g., Envoy proxy sharing app container's network). In Kubernetes, all containers in a Pod share a network namespace using this mechanism (via the pause/infra container). Use case: attach a debug container (`nicolaka/netshoot`) to a running container's network without restarting it.

Q12. How does Docker Compose handle network isolation between projects?

Each Docker Compose project gets its own default network named `projectname_default`. Services in project A cannot reach services in project B by name (different networks). To allow cross-project communication: either connect containers to a shared external network (`networks: myshared: external: true`) or use a specific shared network created outside compose. This provides automatic project-level isolation without any configuration. Override with a custom network name using: `networks: default: name: myapp_network`.

Q13. What is the `docker0` bridge and how is it configured?

`docker0` is the default bridge interface created by Docker on the host. Default subnet: `172.17.0.0/16`. Each container on the default bridge gets an IP from this subnet. `docker0` acts as the gateway (`172.17.0.1`) for all containers. Configure via `/etc/docker/daemon.json: {"bip": "192.168.1.1/24", "fixed-cidr": "192.168.1.0/25"}`. Problem: `docker0` subnet can conflict with existing corporate networks. Always configure custom subnets for production Docker deployments to avoid routing conflicts.

Q14. Explain the difference between Docker networking on Linux vs Mac/Windows.

On Linux, Docker runs natively — containers share the host kernel and network stack. `docker0` bridge is a real Linux bridge. Port-mapped services are accessible at `localhost`. On Mac/Windows, Docker Desktop runs in a VM (HyperKit/WSL2). The Docker network is inside the VM, not on the host. Port-mapped services are accessible at `localhost` via a proxy. `--network host` on Mac/Windows doesn't actually share the physical host

network — it shares the VM's network. This causes confusion when testing network-heavy apps on Mac.

Q15. What are Docker network policies and how do you enforce them?

Docker standalone networking has no built-in network policy (all containers on the same user-defined network can reach each other). To enforce policies: (1) Create separate networks for different tiers (frontend-net, backend-net, db-net). (2) Connect services only to the networks they need access to. (3) For fine-grained policies, use Kubernetes NetworkPolicy (requires a CNI plugin like Calico or Cilium). (4) Use Docker Swarm with network segmentation. (5) Application-level auth (mTLS) as an additional layer.

Q16. How do DNS search domains work in Docker?

Docker sets `/etc/resolv.conf` in containers with: `nameserver 127.0.0.11` (for user-defined networks) or the host DNS (for default bridge). The search domain is set to the network name for user-defined networks. This allows short hostname resolution — containers resolve other containers by short name without specifying the full domain. The search order: `container_name`, `container_name.network_name`, then external DNS. Override with `--dns-search` or in `docker-compose.yml`'s `dns_search` option.

Q17. What is the impact of `iptables=false` in Docker daemon config?

Setting `iptables: false` in `/etc/docker/daemon.json` disables Docker's automatic iptables management. Docker will NOT create NAT rules, FORWARD rules, or DOCKER chains. You must manually manage all networking rules. Use this when: (1) Running Docker behind a firewall that manages iptables. (2) Conflict with another tool that manages iptables (e.g. Kubernetes with its own iptables). (3) Using nftables instead of iptables. Risk: port mapping won't work automatically; containers may have no outbound connectivity.

Q18. How do you limit container network bandwidth?

Docker doesn't have built-in network bandwidth limiting. Options: (1) Use `tc` (traffic control) on the host veth interface: `tc qdisc add dev veth123 root tbf rate 10mbit burst 10mb latency 50ms`. (2) Use CNI plugins in Kubernetes (Calico, Cilium support bandwidth annotations). (3) Use a sidecar proxy (Envoy) to rate limit at the application layer. (4) AWS VPC networking with ECS/EKS supports enhanced networking with predictable bandwidth. (5) Docker Swarm `resource_limits` don't include network (only CPU/memory).

Q19. What is an attachable overlay network?

By default, overlay networks are only usable by Docker Swarm services. An attachable overlay network (`--attachable`) also allows standalone containers (`docker run`) to join the overlay. This enables: testing Swarm services from a standalone container, running debugging tools in the overlay namespace, mixing Swarm services and standalone containers on the same overlay. Create: `docker network create --driver overlay --attachable myoverlay`. Connect standalone: `docker run --network myoverlay myapp`.

Q20. How does Docker handle IPv6 networking?

Docker supports IPv6 via: (1) Enable IPv6 in `daemon.json`: `{"ipv6": true, "fixed-cidr-v6": "2001:db8:1::/64"}`. (2) `docker network create --ipv6` for user-defined networks. (3) Containers get both IPv4 (from bridge) and IPv6 addresses. (4) Port mapping with `-p` works for both IPv4 and IPv6. IPv6 in Docker has limitations: NAT64 for outbound, `iptables6` rules are less polished. In production Kubernetes, IPv6 and dual-stack (IPv4+IPv6) are better supported via CNI plugins.